A Unified Hardware Architecture for Stateful and Stateless Hash-Based Key/Signature Generations

Yechu Zhang*, Yuxuan Chu*, Yaodong Wei, Yueqin Dai, Qiu Shen and Jing Tian(✉)

Nanjing University, Nanjing, China,
{zhangyechu,chuyuxuan,yaodongwei,yq_dai}@smail.nju.cn
{shenqiu,tianjing}@nju.edu.cn

Abstract. Hash-based signature (HBS) schemes, including LMS, XMSS, and SPHINCS+, have become crucial components of post-quantum cryptography. LMS and XMSS are stateful schemes, while SPHINCS+ is stateless, which can be applied in different scenarios. A variety of hash operations in these schemes lead to complex input/output patterns for the hash cores. In this paper, we present an efficient and configurable hardware architecture that supports key generation and signing for all three schemes. Their complex procedural flows are abstracted into 11 shared and parameterized tasks under a unified control module, avoiding controller state blow-up. Driven by hierarchical counters, this approach maximizes resource reuse and preserves scalability, occupying only 17% of the total LUTs. Moreover, the design employs two hash cores with unroll-2 scheduling, which are experimentally validated to strike a favorable balance between area and time. We further introduce an asymmetric dual-path hash input logic (HIL) for each of them: a dedicated parallel lane for the high-frequency One-Time Signature (OTS) task and a flexible padding-shifter for all other tasks. This eliminates wide multiplexers and achieves a superior area-time balance. On Artix-7 FPGA, our unified design occupies 24.2k LUTs/13.7k FFs/16.5 BRAMs. Compared to state-of-the-art single-scheme designs, our architecture achieves up to 4.12 × /10.92× lower Area-Time Product (ATP) for LMS/XMSS signing and 2.47 × /6.61× lower ATP for key generation. More importantly, we provide a flexible, efficient, and scalable hardware foundation for the diverse practical deployments of HBS.

Keywords: Post-Quantum Cryptography · SPHINCS+ · LMS · XMSS · FPGA · Digital Signature

1 Introduction

With the rapid advancement of quantum computing, classical computational problems such as integer factorization and discrete logarithms are expected to be efficiently solved, thereby posing severe security threats to widely deployed public-key cryptosystems such as RSA (Rivest–Shamir–Adleman) [RSA78] and Elliptic Curve Cryptography (ECC) [Por13]. To address this emerging risk, the U.S. National Institute of Standards and Technology (NIST) launched the Post-Quantum Cryptography (PQC) standardization project in 2016 to solicit cryptographic algorithms that can withstand quantum attacks [CCJ+16]. The goal of this initiative is to promote the transition of post-quantum cryptography from

(✉)Corresponding author.
* Both authors contributed equally to this work.

theoretical exploration to practical engineering and to accelerate its global deployment in real-world systems.

According to their mathematical foundations, PQC algorithms can be broadly classified into five categories: lattice-based schemes [NIS23b, NIS23a], code-based schemes [MAB+18], multivariate-based schemes [DS05], isogeny-based schemes [DF17], and hash-based schemes. Among these, only the lattice-based, code-based, and hash-based families have been selected for the NIST standardization process.

In particular, HBS schemes are built solely on standard cryptographic hash functions. As a result, their security rests only on the well-studied properties of these functions, without relying on additional algebraic hardness assumptions, which makes HBS schemes a conservative and robust choice for long-term cryptographic security. HBS schemes mainly include the stateful LMS and XMSS, and the stateless SPHINCS+ (standardized as SLH-DSA), specified in the IETF standards RFC 8554 [MCF19], RFC 8391 [HBG+18], and RFC 7517 [BHK+19], respectively. LMS and XMSS were approved by NIST in 2020 as transitional PQC schemes [CAD+20], while SPHINCS+ was selected in 2023 as a NIST post-quantum digital signature standard [NIS22].

Stateful HBS schemes maintain a private signing state to track key usage, enabling compact signatures and high throughput. However, they are inherently vulnerable to state loss or synchronization rollbacks, which can lead to catastrophic security breaches through key reuse. In contrast, stateless schemes eliminate state-related failure modes, thereby augmenting operational robustness. Nevertheless, this resilience comes at the expense of significantly larger signature sizes and increased computational overhead, as they require more complex hypertree structures to avoid explicit state management. Consequently, a practical deployment often benefits from supporting both paradigms within a single platform, allowing system designers to trade off performance and signature size against operational robustness by selecting the most suitable scheme per device role and threat model [WOS24].

Related Works Although the theoretical foundations of HBS have been established for decades, they have only gained widespread traction recently as the threat of quantum computers has emerged. Initial acceleration efforts primarily focused on software-based platforms [BHRvV21, BDH11, CKRS20]. [BDH11] presents both LMS and XMSS implementations on an ARM Cortex-M4 platform. As research matured, hardware-software co-design solutions emerged to enhance implementation efficiency. In 2019, Wang et al. [WJW+18] completed the hardware-software co-design of an XMSS hardware accelerator on resource-constrained RISC-V embedded processors, employing targeted optimizations across both layers. In 2020, Kumar et al. [KGC+20] introduced a co-design architecture that further increased XMSS computational speeds. In 2024, Saarinen [Saa24] proposed SLotH, a high-efficiency co-design for SPHINCS+, leveraging a RISC-V core to drive secure and high-performance hardware execution. Despite these advancements, conventional co-design architectures often encounter performance degradation during the processing of small packets or frequent handshakes. The substantial communication overhead—often spanning dozens to hundreds of clock cycles, necessitated by SoC bus protocols (e.g., AXI/AHB)—frequently offsets the gains provided by hardware acceleration, constituting a primary systemic bottleneck.

To overcome performance limitations and enhance implementation efficiency, research into HBS hardware accelerators has garnered significant attention. For LMS, Song et al. [SHTW23] designed a complete architecture covering key generation, signature, and verification. While their highly parallel hash core achieved significant speedups, the substantial resource overhead limits its suitability for edge devices. For XMSS, Cao et al. [THG24] optimized node traversal and authentication path computation using Depth-First Search (DFS) and the Buchmann-Dahmen-Schneider (BDS) algorithms, respectively,

demonstrating marked efficiency gains on both FPGA and ASIC platforms. Regarding SPHINCS+, Amiet [ALCZ20] utilized deep pipelining and multi-clock domain designs to achieve high-speed performance, though at the cost of high resource consumption. Conversely, Deshpande et al. [DLK+25] proposed an area-efficient implementation that prioritizes a compact footprint over raw throughput. In 2025, Huang et al. [HLL+25] introduced a reconfigurable SPHINCS+ processor aimed at balancing resource efficiency with operational flexibility.

Most existing works focus on isolated implementations of a single HBS algorithm, while research into integrated, multi-scheme HBS architectures remains limited. Thoma et al. [THG24] integrated LMS and XMSS. And [WOS24] explored hardware-software co-design for the verification stage of LMS, XMSS, and SPHINCS+. Nevertheless, research into a holistic hardware platform supporting all three mainstream HBS schemes is still lacking. We argue that an efficient, unified implementation is crucial to address this research void, enabling a flexible trade-off between performance and security for practical PQC applications.

Contributions We present the first unified hardware architecture that natively supports both stateful (LMS/XMSS) and stateless (SPHINCS+) hash-based signatures within a shared datapath and control framework.

This work focuses on the primary overheads of signing and key generation for the sender side. Because verification is comparatively lightweight and can be efficiently managed by existing receiver-side computational resources, dedicated hardware acceleration for verification is not implemented.

Unlike prior accelerators that target one scheme, our design consolidates the common primitives—OTS chains, Merkle/FORS tree operations, and message-digest generation—into a reusable task-level abstraction, enabling multi-scheme support without structural hardware duplication. Our main contributions are summarized as follows:

- We introduce an efficient and flexible hardware architecture integrating three mainstream hash-based signature schemes—SPHINCS+, XMSS, and LMS—on a single platform for the first time. This architecture, while maintaining area efficiency, can support extremely latency-sensitive real-time PQC migration scenarios, such as high-performance network gateways and high-concurrency hardware security modules.

- To reduce the area overhead associated with supporting multiple schemes, we break down the HBS algorithm procedures into diverse hardware tasks and design configurable computing units. It enables a unified task scheduler and data-flow controller, which in turn maximizes hardware reuse across all three algorithms. The tightly-coupled all-hardware controller we designed (occupying only about 17% of the LUT resources) shortens the control logic response time to 1-2 cycles by eliminating the software scheduling overhead.

- Considering the characteristics of different tasks, we propose a dual-path architecture for hash-input logic consisting of a dedicated high-speed path and a flexible resource-optimized path. For One-Time Signature tasks, a dedicated 1088-bit input register matched to the Keccak rate is designed, enabling single-cycle data loading. For other tasks, a resource-optimized path based on a padding shifter is employed. This asymmetric design ensures optimal performance while preserving input flexibility and minimizing area costs because it eliminates the need for large multiplexers.

- The design was coded in SystemVerilog and implemented on an Artix-7 Xilinx FPGA using Xilinx Vivado 2018.3. Experimental results demonstrate that the proposed architecture achieves a balanced area-time trade-off across the three HBS algorithms.

The LMS and XMSS implementations surpass existing works in speed and ATP. Furthermore, its area is much smaller than that of the three schemes combined, validating the effectiveness and feasibility of the proposed design.

2 Preliminaries

2.1 Winternitz One-Time Signature (WOTS)

As a member of the one-time signature (OTS) family, the WOTS scheme imposes a fundamental constraint: each private key must be used for signing only once. If the same key is reused to sign two distinct messages, the security of the scheme becomes severely compromised. The main parameters of a WOTS scheme are as follows:

- n: the output length of the underlying hash function in bits, which also determines the security level of the scheme.
- Winternitz parameter w: controls the length of the hash chains used in the OTS signing process, i.e., the number of hash iterations. A larger w produces shorter signatures but slows down key generation, signing, and verification, whereas a smaller w increases signature length but accelerates computation. Therefore, w allows a trade-off between signature size and computational efficiency.
- len: the number of hash chains used for signing a message.

2.1.1 Hash Chain Function

Both public key generation and signature computation in WOTS rely on hash chains. For Public Key Generation, each private key element $\mathsf{wots.sk_i}$ is iteratively hashed $2^w - 1$ times to produce len intermediate public key elements $\mathsf{wots.pk_i}$. These elements are then compressed into a single n-bit public key $\mathsf{wots.pk}$. For signature generation, the number of hash iterations applied to each private key element is determined by the message M. The process begins by computing a message digest of length n. This digest is divided into len_1 blocks, digest_i, each of at most w bits. A checksum is then calculated as $\mathsf{checksum} = (2^w - 1) \cdot len_1 - \sum \mathsf{digest}_i$ to prevent forgery by manipulating the message. This checksum is subsequently split into len_2 blocks, csum_i. The combined sequence of $len = len_1 + len_2$ blocks forms the processed message: $\mathsf{msg} = \mathsf{digest}_i \parallel \mathsf{csum}_i$. Each private key element $\mathsf{wots.sk_i}$ then undergoes a chain of msg_i hash iterations to produce the signature element $\mathsf{wots.sig_i}$. The final WOTS signature $\mathsf{wots.sig}$ is formed by concatenating all len of these elements, as shown in Equation (1).

$$\mathsf{wots.sig} = \mathsf{wots.sig_0} \parallel \mathsf{wots.sig_1} \parallel \cdots \parallel \mathsf{wots.sig_{len-1}} \tag{1}$$

2.2 Merkle Tree

Considering the large size of public keys, directly concatenating a large number of WOTS public keys to form a single long-term public key is highly impractical. Consequently, HBS introduces the Merkle tree, leveraging a balanced binary tree structure to efficiently generate a smaller and more secure long-term public key. Each node of the Merkle tree is an n-bit string, and the leaf nodes are derived from WOTS public keys. A Merkle tree of height h contains 2^h leaf nodes and can be used to sign 2^h distinct messages, enabling secure multiple-time signatures. Leaf nodes are paired and hashed to form the first layer of tree nodes, which are subsequently paired and hashed to generate the next layer. This process continues iteratively until a single hash value—the root node—is obtained, representing the final long-term public key.

The Merkle tree authentication path is a critical component in LMS/XMSS/SPHINCS+ signatures. It consists of the sibling nodes along the path from the signing leaf node (Sign) to the root node (Root), as illustrated in Figure 1. The verifier computes the hash of the signing leaf node and iteratively combines it with the nodes in the authentication path to reconstruct the root node. Finally, the verifier compares the computed root with the public key, thereby verifying the existence and integrity of the data without accessing the entire dataset.

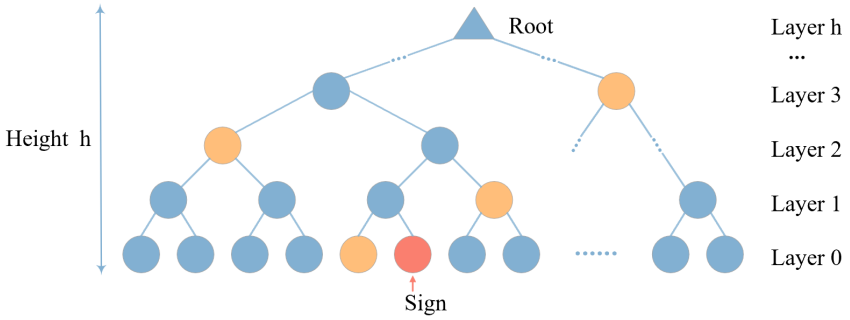

Figure 1: Authentication Path of Merkle Tree. The red node represents the leaf node used to sign, and the yellow nodes constitute the corresponding authentication path.

2.3 LMS

LMS consists of WOTS and a Merkle tree, as shown in Figure 2. Each leaf node of the Merkle tree corresponds to *len* hash chains of the WOTS. Each hash chain starts from the corresponding WOTS private key and undergoes *len* iterative hash computations. The resulting *len* hash values are further hashed to compress into the corresponding WOTS public key, which forms the leaf node of the Merkle tree.

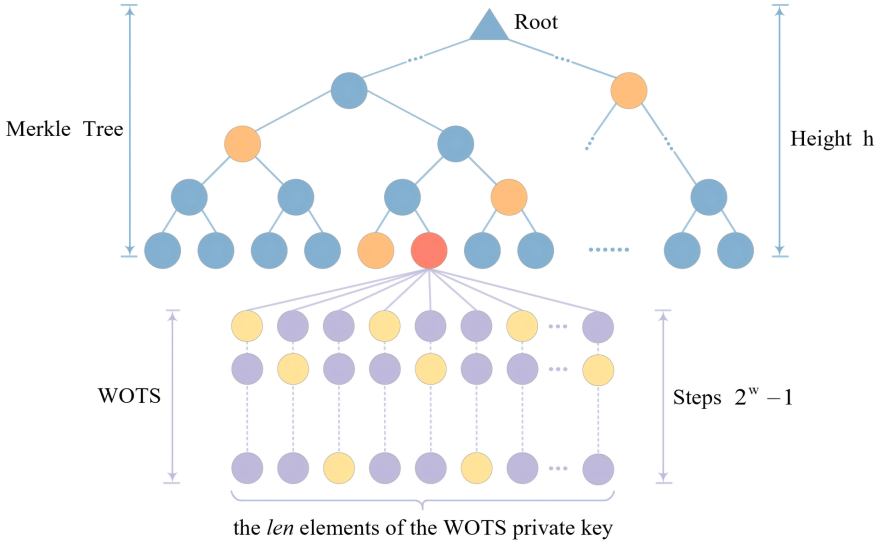

Figure 2: LMS overall structure. The purple and yellow nodes represent the hash nodes and signature nodes of WOTS, respectively, while other colors of nodes are consistent with Figure 1.

2.3.1 LMS Signature

The LMS signature, as shown in Equation (2), comprises five components: the type, a random string R, the OTS signature wots.sig, the leaf index q, and the Merkle tree authentication path path. The type is determined by the selected LMS parameters, while wots.sig and path are derived from and correspond to the specific leaf node indexed by q.

$$\mathsf{LMS.sig} = \mathsf{type} \parallel R \parallel \mathsf{wots.sig} \parallel q \parallel \mathsf{path} \tag{2}$$

2.3.2 Prefix

To ensure the security of hash computations, LMS incorporates a prefix into the hash input. The prefix typically consists of a key identifier I, a signature index q, and fixed constants used to distinguish different hash operations.

2.4 XMSS

The overall structure of XMSS, as illustrated in Figure 3, is a hierarchical construction comprising, from the bottom up, a variant of WOTS called WOTS+, L-trees, and a Merkle Tree. Notably, XMSS introduces an L-tree to replace the single hash operation used in LMS for compressing the WOTS+ public key. The L-tree has $\lceil \log_2 len \rceil$ layers, with its leaf nodes being the len WOTS+ public key elements, and its root node serving as the corresponding leaf node in the Merkle Tree. Since len may be odd, the L-tree is an unbalanced binary tree. If the number of nodes at a level is odd, the last node is moved upward until it pairs with another node to perform hash computation for the next level.

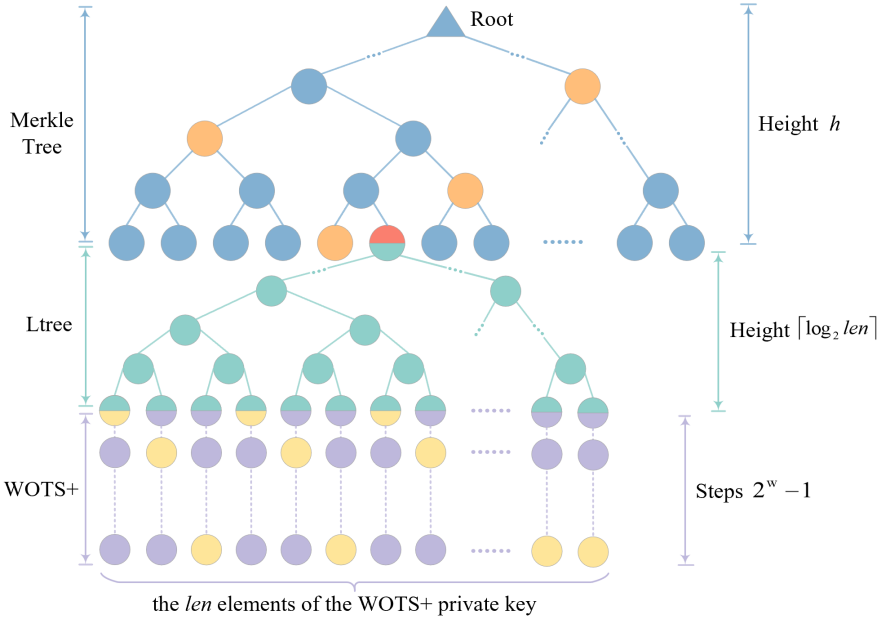

Figure 3: XMSS overall structure. Cyan nodes are assigned to the L-tree's hash nodes, while the colors for all other node types are consistent with Figure 2.

2.4.1 XMSS Signature

The XMSS signature, as shown in Equation (3), consists of four parts: signature index idx.sig, random string R, WOTS+ signature, and the Merkle tree authentication path.

Given that the WOTS+ signature shares the same structural format as the WOTS signature, the notation wots.sig is used generically throughout this paper to represent both.

$$\mathsf{XMSS.sig} = \mathsf{idx.sig} \parallel R \parallel \mathsf{wots.sig} \parallel \mathsf{path} \tag{3}$$

2.4.2 Prefix and Bitmask

The architectural differences between LMS and XMSS primarily manifest in their input preparation for the hash function. For initial message compression, both schemes append a prefix to the input; however, the XMSS prefix is more complex, incorporating an n-bit random string R, the n-bit tree root, and a 4-byte signature index idx_{sig}. For tree node computation, LMS simply concatenates the prefix with the node data. In contrast, XMSS implements a randomized hashing strategy: it generates a unique KEY and bitmask BM based on the current address $ADRS$ and a public $SEED$. The node data must be XORed with BM before being concatenated with KEY for the final hash. If multiple data blocks are present, separate bitmasks are generated and XORed individually. The independent derivation of masks and prefixes in XMSS involves a more complex hashing sequence than in LMS. Consequently, XMSS offers superior resistance to certain collision attacks at the cost of reduced throughput.

2.5 SPHINCS+

SPHINCS+ is structured as a hypertree composed of FORS (Forest Of Random Subset) and WOTS+ components, as illustrated in Figure 4. The signature process initiates at the bottom, where a message digest is signed by FORS. The resulting FORS root is then signed by the WOTS+ key pair of the bottom XMSS layer (Layer 0). This hierarchical signing continues upward, where the root of each XMSS tree at layer i is signed by a WOTS+ instance at layer $i+1$, eventually reaching the global root.

2.5.1 FORS

FORS can be viewed as K parallel Merkle trees of identical height. The roots of K trees are hashed together to generate the final n-bit FORS root (public key). The FORS signature consists of the signatures of all K subtrees, each $\mathsf{sig.subtree_i}$ comprising the subtree's private key $\mathsf{sk.fors_i}$ and authentication path $\mathsf{fors.path_i}$, as expressed in Equation (4).

$$\begin{aligned} \mathsf{FORS.sig} &= \mathsf{sig.subtree_0} \parallel \cdots \parallel \mathsf{sig.subtree_{k-1}} \\ &= \mathsf{sk.fors_0} \parallel \mathsf{fors.path_0} \parallel \cdots \\ &\quad \parallel \mathsf{sk.fors_{k-1}} \parallel \mathsf{fors.path_{k-1}} \end{aligned} \tag{4}$$

2.5.2 SPHINCS+ Signature

As formalized in Equation (5), the SPHINCS+ signature consists of three parts: a random string R, a FORS signature FORS.sig, and a hypertree signature HT.sig.

$$\mathsf{SPHINCS+.sig} = R \parallel \mathsf{FORS.sig} \parallel \mathsf{HT.sig} \tag{5}$$

The hypertree signature HT.sig aggregates authentication data from all D layers, each contribution being an XMSS signature $\mathsf{XMSS.sig_i}$ that includes a WOTS+ signature $\mathsf{wots.sig_i}$ and a Merkle path $\mathsf{path_i}$, shown in the following Equation (6):

$$\mathsf{HT.sig} = \mathsf{XMSS.sig_0} \parallel \cdots \parallel \mathsf{XMSS.sig_{D-1}} \tag{6}$$

where $\mathsf{XMSS.sig_i} = \mathsf{wots.sig_i} \parallel \mathsf{path_i}$ for layer i.

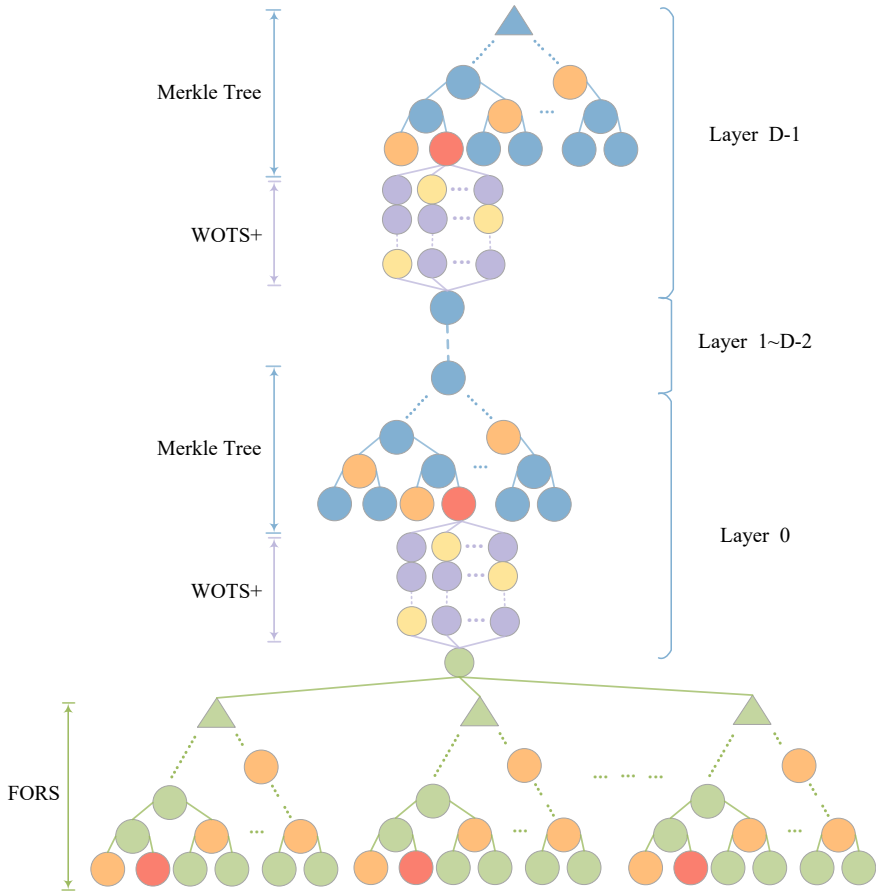

Figure 4: SPHINCS+ overall structure. The green nodes denote the hash nodes within the FORS Tree, with all other node colors established in Figure 2.

2.6 Parameter Sets

LMS, XMSS, and SPHINCS+ support standardized hash functions, including SHA-2 and SHAKE256[NIS22, CAD+20]. The summary of parameters for these schemes is shown in Table 1 and Table 2, where all the security levels are included.

2.7 Commonality Analysis

Based on the above descriptions, the three signature schemes exhibit significant similarities. On one hand, all underlying operations rely solely on hash functions, which implies that in hardware implementations, the hash module can be shared across different schemes without conflict. On the other hand, from a structural perspective, all three schemes rely on OTS and Merkle trees as fundamental components, with their signatures comprising OTS signatures and Merkle tree authentication paths. This structural commonality suggests that these components can be reused across schemes, which not only reduces hardware overhead but also significantly enhances area-efficiency.

Table 1: Parameters Set in XMSS/LMS

Scheme	n	w	len	h
XMSS	32	2	133	{10, 16, 20}
	32	4	67	{10, 16, 20}
LMS	32	1	265	{5, 10, 15, 20, 25}
	32	2	133	{5, 10, 15, 20, 25}
	32	4	67	{5, 10, 15, 20, 25}
	32	8	34	{5, 10, 15, 20, 25}

Table 2: Parameters Set in SPHINCS+

Scheme	n	len	ω	k	a	h	d
SPHINCS+128-f	128	35	4	33	6	3	22
SPHINCS+192-f	192	51	4	33	8	3	22
SPHINCS+256-f	256	67	4	35	9	4	17
SPHINCS+128-s	128	35	4	14	12	9	7
SPHINCS+192-s	192	51	4	17	14	9	7
SPHINCS+256-s	256	67	4	22	14	8	8

3 Proposed Hardware Architecture

3.1 Top-Level Architecture

To achieve efficient and flexible hash-based signature processing, we have constructed a task-based unified hardware architecture. The core idea of this architecture is to decompose the SPHINCS+, XMSS, and LMS signature algorithms into a series of coarse-grained hash tasks, which are scheduled by a unified controller while maximizing hardware resource reuse.

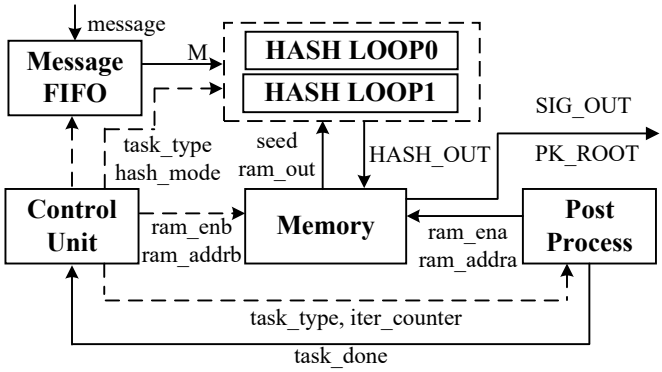

Figure 5: Overall hardware architecture.

The top-level architecture for our design is shown in Figure 5. The main blocks are:

1. **HASH LOOP0/1:** Two parallel hash engines that process **M**, seed, and other input components, then write back the results as *hash_out* to memory.

2. **Control Unit:** Manages the internal counters related to task control, issues control signals to subordinate modules, and drives the memory read port (ram_enb, ram_addrb).

3. **Memory:** Two dual-port memories are adopted to decouple the Control Unit's read operations from the hash engines' write-back cycles, effectively eliminating structural hazards. These two memories are dedicated to intermediate node storage and signature result buffering, respectively. Consequently, results from signing or key generation (SIG_OUT, PK_ROOT) are also temporarily buffered here before being streamed out.

4. **Message FIFO:** Buffers incoming messages and streams the word sequence **M** to the hash datapath in 64-bit words.

5. **Post Process:** Receives *iter_counter* from the Control Unit to determine task completion and asserts *task_done*. Concurrently, it drives the memory write port (ram_ena, ram_addra) based on task type and counter signals.

More details about them are shown below.

3.2 HASH LOOP

The HASH LOOP modules constitute the computational core of the accelerator, consisting of two engines operating in parallel, as depicted in Figure 6.

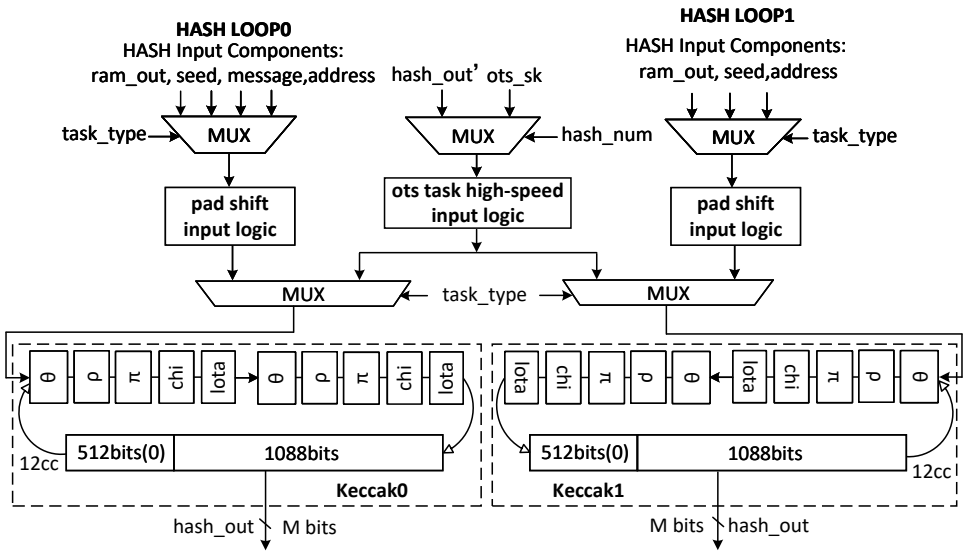

Figure 6: Architecture of the HASH LOOP module.

3.2.1 Keccak Core

Our architecture supports SHAKE256-based HBS schemes. Specifically, the hash engine in the HASH LOOP module is a highly optimized iterative unit that integrates a Keccak-p[1600] core with an unrolling factor of 2 and a hash-input logic (HIL) that supports both 64-bit streaming and rate-wide (1088-bit) loading modes.

Rationale for SHAKE-Only Support In our design, we exclusively support SHAKE256 as the underlying hash function for HBS. While SHA-2 remains a common alternative, we prioritize SHAKE256 for the following three reasons:

Table 3: Measured slice, time, and ATP values under different parallelism and unrolling factors for LMS
(len=133, ω=2, h=10)

P	$U=1$			$U=2$		
	SLICE	TIME	ATP	SLICE	TIME	ATP
1	7601.41	80.36	61.09	8395.66	49.85	41.85
2	10807.48	50.40	54.47	11601.73	35.03	**40.64**
3	13647.41	40.20	54.86	14838.79	29.88	44.34
4	16487.35	35.43	58.41	18075.85	27.63	49.94
6	22167.23	30.44	67.48	24549.98	25.16	61.78
8	27847.10	27.71	77.16	31024.10	23.81	73.86

[a] The row (P) represents the degree of parallelism, and the column grouping (U) represents the expansion series.

- **Security Strength**: SHAKE256 and SHA-2, when instantiated with 256-bit outputs, provide comparable security: about 128-bit collision resistance and 256-bit preimage resistance.
- **Performance Advantage**: SHAKE256 provides superior throughput due to its larger block size (rate) and a more hardware-friendly sponge construction. Compared to the Merkle-Damgård (MD) construction of SHA-2, the sponge structure exhibits shorter critical paths in high-performance implementations.
- **Engineering Synergy**: Our hardware stack—including the low-level keccak core, the parallelized HASH LOOP modules, and the specific parameter sets for SPHINCS$^+$, XMSS, and LMS—is deeply co-optimized for the SHAKE256 structure. This unified engineering approach maximizes resource reuse and minimizes control overhead across different HBS schemes.

Analysis of the "dual-parallel, unroll-2" Architecture The three target HBS algorithms exhibit a fundamental computational dichotomy: Parallelizable Tasks (PTs), such as Merkle node generation across the same layer, and Data-Dependent Tasks (DDTs), such as message digesting and hash computation within a single OTS chain. These tasks impose conflicting optimization priorities: PTs demand high-throughput via parallelized hash cores to maximize concurrent processing, whereas DDTs necessitate minimized execution latency (fewer clock cycles) due to their inherent sequential dependencies ($Hash(Hash(\dots))$), which render traditional pipelining ineffective for single-chain acceleration. Therefore, we propose the "Dual-Parallel, Unroll-2" architecture as a strategic tradeoff. Compared to a baseline single hash core, it can achieve about 4× and 2× faster speed for PTs and DDTs, respectively. We adopt the parallelism + unrolling strategy over interleaved pipelining for two reasons. First, pipelining prioritizes aggregate throughput at the cost of single-task latency. Since HBS protocols involve strict sequential dependencies (DDTs), our unroll-2 architecture is superior as it directly compresses the total cycle count, minimizing the latency required for DDTs. Second, interleaved pipelining incurs substantial area overhead due to the numerous registers needed for intermediate states, which contradicts our resource-efficient goal.

Regarding the selection of the unrolling factor (U) and parallelism level (P), we justify our design by evaluating the ATP of LMS with len=133, ω=2, and h=10 under various configurations. Although only one LMS parameter set is reported, it is representative in that it follows the standard HBS workflow and exercises the dominant operations and memory-access patterns of the stateful HBS family, including frequent OTS-related

hashing and extensive intermediate-state handling. Specifically, we examine parallelism levels $P \in \{1, 2, 3, 4, 6, 8\}$ and unrolling factors $U \in \{1, 2\}$. The rationale for limiting U to 2 is rooted in timing constraints: at $U = 2$, the critical path resides within the control logic; however, the $U = 3$ configuration shifts this path into the Keccak core logic. Consequently, any further unrolling would significantly extend the critical path, leading to a substantial degradation in operating frequency.

Experimental results in Table 3 demonstrate that the proposed architecture achieves an optimal balance between performance and resource efficiency when $U = 2$ and $P = 2$. First, the $U = 2$ configuration effectively mitigates the sequential bottlenecks inherent in DDTs by executing two rounds of operations within a single clock cycle, resulting in a substantial reduction in the time metric compared to $U = 1$. Second, regarding the degree of parallelism, as shown in Figure 7, under $U = 2$, the ATP first decreases with increasing parallelism and reaches its minimum at $P = 2$, after which further increasing P leads to a monotonic ATP degradation due to rapidly growing resource overhead. Therefore, $(P, U) = (2, 2)$ is selected as the optimal design point, achieving the best trade-off between performance gain and hardware cost.

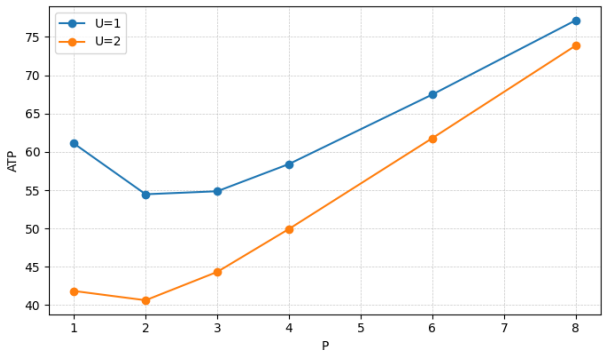

Figure 7: ATP across parallelism under unrolling factor 1 and 2 for LMS (len=133, ω=2, h=10)

3.2.2 Hash Input Logic

Implementing HBS hardware necessitates a trade-off between the high-throughput requirements of OTS chains—which account for over 80% of the total workload—and the flexibility demanded by diverse tasks such as message compression and tree computation. This challenge is further intensified when unifying SPHINCS+, XMSS, and LMS, as a universal flexible path would introduce excessive latency, while fully dedicated paths for every task would lead to prohibitive area overhead and reduced clock frequency. To resolve this, we propose an asymmetric dual-path Hash Input Logic (HIL) as shown in Figure 8. By decoupling the execution into a specialized high-speed channel for OTS operations and a resource-efficient, versatile lane for miscellaneous tasks, our design achieves an optimal balance between processing speed and hardware economy.

Dedicated high-speed path for OTS For the OTS chain tasks, which dominate the computation, the hash input data format is fixed and constructed by concatenating multiple components. To achieve maximal data feeding efficiency, we implemented a dedicated 1088-bit wide OTS input register for OTS chain tasks, which matches the Keccak rate. This allows all components for a single hash absorption operation to achieve a single-cycle input supply. Furthermore, to maximize resource utilization, the two parallel Keccak

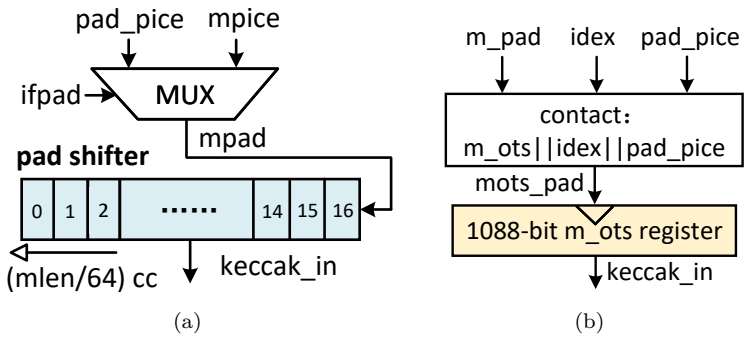

Figure 8: Proposed logic architectures: (a) Dedicated high-speed path for OTS. (b) Flexible resource-efficient path for other tasks.

cores share a common 1088-bit input register. To avoid resource contention and simplify control overhead, we adopted a deterministic, staggered start control strategy: in each OTS computation cycle, Keccak0 is always started first, followed by Keccak1 exactly one clock cycle later, as depicted in Figure 10. This approach enables efficient collaboration between the dual cores without the need for complex arbitration logic or extra register resources. Consequently, this design achieves significant area savings and streamlined control logic at the expense of a negligible performance penalty.

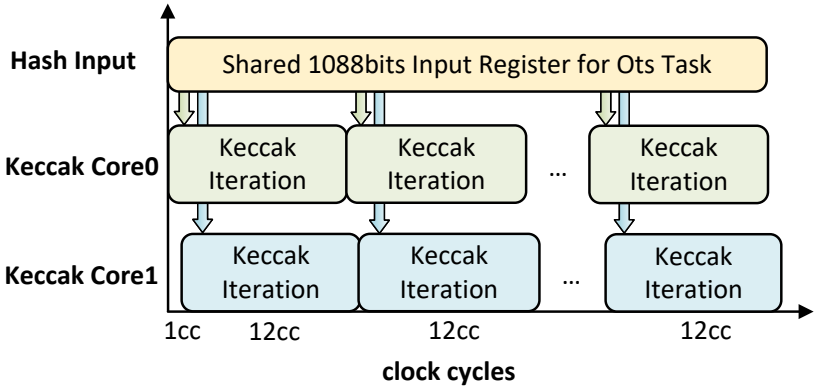

Figure 9: Timing of OTS chain.

Flexible resource-efficient path for other tasks For the remaining tasks and arbitrary-length inputs like message M', which account for a minor fraction of the execution time but are diverse in input length, our goal is to achieve maximum flexibility with minimal area overhead. The motivation behind this approach is to avoid the excessive hardware overhead associated with a unified high-throughput architecture. If all diverse tasks were integrated into the primary high-speed data path, a high-fan-in multiplexer would be required at the input of the 1088-bit state register to handle various data alignment combinations. This would lead to prohibitive logic resource consumption and severe routing congestion. Therefore, we implement a versatile padding unit that bypasses the area-heavy static multiplexers typically required for variable-length data. The unit performs dynamic suffix padding controlled by the configuration signals *pad_pos* and *is_last_blk*. Furthermore, a 64×17-bit buffer is employed to manage serial-to-parallel absorption. Upon detecting the final block ($is\_last\_blk = 1$), padding is appended from the position

$pad\_pos = input\_length\%8$ (in bytes) within the 64-bit input block, and continues until the valid data length reaches the rate $r = 1088$. Since the Keccak iteration time (12 cycles) is comparable to the buffer shift time (17 cycles), the resulting latency mismatch introduces intermittent pipeline bubbles. Although this design choice incurs a marginal performance penalty, the architectural advantages are substantial: it facilitates timing closure, reduces logic resource utilization, and alleviates routing congestion. This trade-off is well-justified given that these ancillary operations constitute less than 20% of the total execution time.

3.3 Control Unit

To support the SPHINCS+, XMSS, and LMS algorithms within a unified hardware framework, the controller design faces two major challenges: an extensive state space and a low degree of logic reuse. Adopting a direct implementation by developing separate control logic for each algorithm would lead to severe resource redundancy and prohibitive design complexity. To address this, we propose a parameterized control reuse architecture based on "task abstraction", which achieves flexible support for multiple algorithms with minimal hardware overhead.

Table 4: Task-based abstraction for Hash-Based Signatures.

TASK	**H**	**Algorithm**	**Usage**	**Input Length**
task0	1 0	SPHINCS+ LMS/XMSS	Pseudorandom Number	2N+Mesg Size N+32
task1	2	SPHINCS+/ LMS/XMSS	Message Digest	3N+Mesg Size
task2	5	SPHINCS+	FORS Private Key/Leaf	2N+32
task3	0	LMS/XMSS	OTS Private Key Seed	N+32
task4	7	SPHINCS+	FORS Merkle Tree Node	3N+32
task5	8	SPHINCS+	FORS Tree Public Key(H_l)	(1+k)*N+32
task6	5 0	SPHINCS+ LMS/XMSS	WOTS Private Key	2N+32 N+32
task7	3 4	SPHINCS+/LMS XMSS	WOTS Chain WOTS+ Chain(mask)	2N+32 N+32
task8	8	SPHINCS+/LMS	WOTS Public Key(H_l)	(1+L)N+32
task9	6	XMSS	WOTS+ Public Key(L-tree)	3*(N||32),3N
task10	7 6	SPHINCS+/LMS XMSS	Merkle Tree Node Merkle Tree Node(mask)	3N+32 3*(N||32),3N

3.3.1 Cross-Algorithm Hardware Reuse Based on Task Abstraction

Although the three algorithms differ in their top-level flows, their underlying computational patterns share significant commonalities. For instance, they all employ OTS schemes (WOTS and WOTS+) and all utilize the Merkle tree to aggregate vast quantities of OTS public keys. Based on this insight, we employed a reused, coarse-grained task abstraction control strategy. We abstract and encapsulate the various computation stages from different algorithms—such as OTS chain computation, Merkle tree node updates, FORS tree generation, and message digest generation—into a series of parameterized tasks.

Each abstracted task is parameterized by the top-level controller, translating into a deterministic configuration of internal datapaths and Hash Input Logic (HIL) modes. This dynamic reconfiguration enables a single physical hardware module to be time-multiplexed across SPHINCS+, XMSS, and LMS, thereby supporting diverse computational primitives without the need for structural hardware alterations. This high degree of reuse not only maximizes the utilization of hardware resources but also enhances the scalability of the control architecture. To accommodate new variants of hash-based signatures in the future, only new task sequences and parameter configurations will need to be added to the controller, obviating the need for large-scale hardware modifications to the underlying datapath. Table 4 delineates the core tasks and their attributes, clearly illustrating this task-based abstraction method.

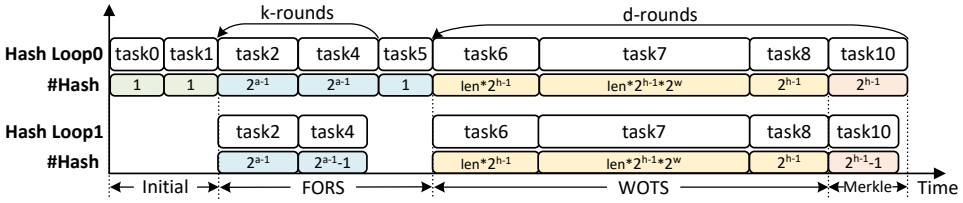

Figure 10: Execution flow and task scheduling of SPHINCS+ signature generation.

To clearly demonstrate the control logic, Figure 8 illustrates the execution flow and computational workload scheduling for SPHINCS+ signature generation as an example. The process begins with sequential hash operations to derive a pseudo-random value (task0) and its corresponding message digest (task1). Subsequently, the k subtrees of the FORS structure are processed. Specifically, task2 parallelizes the generation of 2^a leaf nodes, while task4 computes the remaining internal nodes. Given that each FORS subtree requires $2^a - 1$ hash operations in task4, while Hash Loop0 executes 2^{a-1} iterations, Hash Loop1 only performs $2^{a-1} - 1$ iterations, idling during the final cycle to ensure timing synchronization. Once the k FORS subtree roots are generated, they are aggregated to form the definitive FORS root via the H_l function (task5).

The workflow then enters the Hypertree stage, where d layers of XMSS signatures are computed iteratively, involving WOTS+ computation (task6–8) and Merkle tree updates (task10). Specifically, task8 employs the H_l hash function to compress WOTS+ public keys into XMSS leaf nodes. Mirroring the scheduling in task4, task10 manages the Merkle tree node calculations with a similar idling mechanism to accommodate the odd iteration count. The controller repeats the stage for all d layers to finalize the SPHINCS+ computation.

3.3.2 Hierarchical Counter–Based Control Logic

When integrating multiple complex cryptographic algorithms, conventional monolithic Finite State Machines (FSMs) often encounter the state-space explosion problem. This results in excessively intricate state transition logic that is difficult to design, verify, and scale. To mitigate this complexity, we employ a hierarchical counter-based architecture to achieve systematic orchestration and control of the algorithmic processes.

This approach decomposes complex execution flows into nested looping structures. The hierarchical counter configuration within a single functional task is illustrated in Figure 11. The specific functionalities are defined as follows:

- Bottom-level Counter (*cnt*): Manages hash iterations within a single node. For task7 and task10 in XMSS requiring mask calculations, the maximum iterations are 3 and 4 (i.e., A is set to 2 and 3, respectively). For tasks independent of mask operations, only a single hash is performed ($A = 0$).

- Mid-level counter (*num*): Tracks and controls the node index within a specific hierarchical level, such as the node index of a single OTS chain in task7 or the Merkle tree node index of the current layer in task10.
- Top-level counter (*index*): Controls the high-level execution stage, such as the OTS chain index in task7 or the layer index of the Merkle tree in task10.

In terms of functionality, this multi-level counter hierarchy is fully equivalent to a deterministic finite-state machine (FSM). Its key advantage lies in decomposing a monolithic global state into a set of independent, logically decoupled local states.

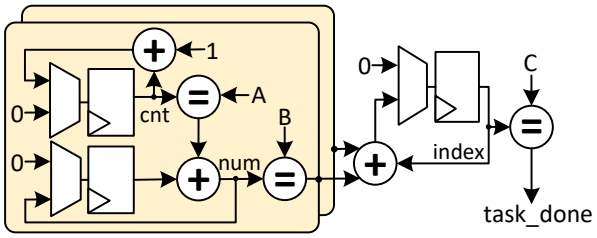

Figure 11: Architecture of hierarchical counter.

3.4 Memory

The total memory footprint of this design is approximately 72 KB, utilizing 16.5 dual-port BRAMs in total. Of these, two BRAMs buffer signatures (SigRAM), and the remaining 14.5 BRAMs store intermediate data (DataRAM). To support LMS/XMSS key generation—which requires storing the entire Merkle tree with maximum height $h = 10$—the 256-bit-port DataRAM is configured with a maximum depth of $2^{(10+1)} = 2^{11}$. Accordingly, the DataRAM is configured as 256×2^{11}, which synthesizes to 14.5 BRAMs. For signature buffering, we adopt a segmented streaming strategy. The maximum signature length per tree is 143×256 bits, requiring a depth of 143; therefore, a SigRAM configured as 256×2^8 suffices and synthesizes to 2 BRAMs. What's more, BRAM read/write operations and the hash computations in the HASH LOOP are scheduled in different pipeline stages, enabling parallel execution. By overlapping memory access with computation, the memory controller prefetches data for the subsequent cycle while the hash core processes the current block, effectively hiding memory access latency.

To cope with limited on-chip storage and the intrinsically large signature-size overhead of SPHINCS+, we employ a segmented signature streaming strategy. The core principle of this strategy is to produce and output immediately. During the signing phase, upon the completion of each computational segment, such as a single FORS tree or a single Merkle tree, its result is immediately dispatched via the I/O interface, rather than awaiting the full signature generation. This streaming approach effectively reduces the peak storage requirement from full-signature buffering to only buffering the current computational segment, significantly conserving BRAM resources while also shortening the system's response latency.

4 Results And Comparisons

4.1 Results

The hardware architecture proposed in this paper is implemented using Xilinx Vivado 2018.3. For fair comparison, the target device selected is an Artix-7 series xc7a200t FPGA.

The proposed architecture achieves a maximum clock frequency of 108 MHz. Detailed performance metrics—including clock cycle counts and total execution latency for key generation (Keygen) and signing (Sign)—are summarized in Table 5.

Table 5: Latency of processor.

Algorithm	**Param**	**Keygen(CCs/ms)**	**Sign(CCs/ms)**
SPHINCS+	$n = 128$	23,688/0.22	537,636/4.95
	$n = 192$	34,687/0.32	926,398/8.52
	$n = 256$	91,510/0.84	1,929,154/17.75
LMS	$\omega = 2$	3,808,440/35.04	3,572/0.03
	$\omega = 4$	5,843,438/53.76	5,429/0.05
XMSS	$\omega = 2$	7,829,494/72.03	7,334/0.07
	$\omega = 4$	15,682,342/144.28	14,549/0.13

For SPHINCS+, with security parameters in $n \in \{128, 192, 256\}$, the Keygen/Sign execution times are 0.22/4.95 ms, 0.32/8.52 ms, and 0.84/17.75 ms, respectively. Because the number of hash invocations is fixed for a given parameter set, the Keygen/Sign latency is constant and does not vary with the public or private key values.

For LMS/XMSS with $n = 256$ and $h = 10$, the Keygen/Sign times for LMS with $w \in 2, 4$ are 35.04/0.03 ms and 53.76/0.05 ms, respectively. For XMSS with $w \in \{2, 4\}$, the times are 72.03/0.07 ms and 144.28/0.13 ms. The signing procedures for LMS and XMSS rely on an OTS scheme where the number of hash computations in the Winternitz chains depends on the message digest being signed, so the signing latency varies within a bounded range. The signing times reported in Table II are the average values obtained over 1000 random signing operations to reflect expected real-world performance.

Table 6: Resource utilization by module.

Modules	**LUT**	**FF**	**BRAM**
Control unit	**4,222/17.4%**	**5,137/37.8%**	**16.5**
Post Process	**895/3.7%**	**1,090/8.0%**	**0**
Hash LOOP	**19,067/78.4%**	**7,432/54.8%**	**0**
⌞ HASH LOOP0	9,657	3,717	0
⌞ HASH LOOP0.HIL	3,303	2,078	0
⌞ HASH LOOP0.Keccak	6,354	1,639	0
⌞ HASH LOOP1	9,410	3,715	0
⌞ HASH LOOP1.HIL	3,056	2,076	0
⌞ HASH LOOP1.Keccak	6,354	1,639	0
Sig RAM	**0**	**0**	**2**
Data RAM	**0**	**0**	**14.5**
Total	**24,184**	**13,659**	**16.5**

Note: LUT/FF percentages are normalized to the total resource usage of this design (the "Total" row).

Table 6 presents the resource consumption of each module on the Artix-7 FPGA. The overall design utilizes 24,184 LUTs, 13,659 FFs, 16.5 BRAMs, and 0 DSPs. The Control Unit consumes 4,222 LUTs (17.4%) and 5,137 FFs (37.8%); its FF share exceeds 30% of the design because it integrates complex logic with numerous counters and state machines (e.g., the ADRS generator and task-control unit), buffers diverse inputs for the two HASH LOOP modules, and maintains a 1088-bit m_{ots} register. The Post-Process unit—responsible for handshaking, task-status management, and precise BRAM writes—uses 895 LUTs (3.68%)

and 1,090 FFs (8.03%). The HASH LOOP module is the core computational engine and the primary resource consumer, requiring 19,067 LUTs (78.4%) and 7,432 FFs (54.8%); this concentration highlights the computation-centric simplicity of our architecture. To further optimize resources, we employ an asymmetric design: HASH LOOP0 executes both parallel and serial tasks, whereas HASH LOOP1 handles only parallel tasks; their utilizations are 9,657/9,410 LUTs and 3,717/3,715 FFs, respectively.

As summarized previously, this design utilizes a total of 16.5 BRAMs. Among them, 14.5 BRAMs are dedicated to storing the extensive amount of intermediate data generated during the execution of the SPHINCS+, XMSS, and LMS algorithms, including but not limited to the nodes of each layer of the Merkle tree and OTS public keys. The remaining 2 BRAMs are used exclusively as a signature buffer.

4.2 Comparisons With Related Works

To objectively position our design relative to the state of the art, we conduct quantitative comparisons against several advanced hardware implementations; results are summarized in Table 7. All results of the table were obtained on Artix-7 FPGA, except for [CWW+22], which was implemented on Virtex UltraScale+ FPGA. As the first hardware implementation to support both LMS and XMSS, and as the newest and most effective full implementation of XMSS, [THG24] uses SHA256 as the core hash function. It introduces a BDS algorithm with configurable parameter k. Compared to their most high-performance configuration with 8 hash cores and 7 WOTS+ chaining modules, our design exhibits speedup of 4.34× (LMS) and 8.97× (XMSS) for Sign with $\{h, \omega\} = \{10, 4\}$. The speedup in Keygen is 2.60×(LMS)/5.43×(XMSS). In terms of ATP, the design in [THG24] is 4.13× (LMS)/10.92× (XMSS) higher than this work for Sign and 2.47× (LMS)/6.61× (XMSS) higher for Keygen. Our design's superior performance stems from two synergistic optimizations. First, the adoption of SHAKE256 provides a higher throughput baseline, as its larger rate (1088 bits) and sponge-based structure are more hardware-friendly than SHA-256. Second, our novel dual-path architecture provides single-cycle input latency for the OTS chain computations, a key performance bottleneck. Notably, we compare our SHAKE256-based design with prior SHA-256 implementations because, to the best of our knowledge, no other high-performance unified hardware architectures for XMSS and LMS currently leverage the SHA-3 family.

Apart from [THG24], the remaining works implement a single signature scheme and are analyzed separately. For SPHINCS+, the fastest existing hardware implementation is [ALCZ20]. They implemented a 12-stage Keccak permutation along with substantial register insertion to minimize the critical path. As a result, their Keccak core can operate at a maximum frequency of 500 MHz. Additionally, a multi-clock-domain scheme was designed, allowing the peripheral control circuit to function at 250 MHz. Compared to [ALCZ20], our design exhibits a higher latency for individual SHAKE256 operations due to our emphasis on resource efficiency. However, our solution demonstrates a substantial resource advantage, reducing LUT/FF/DSP consumption by 49.61%/81.16%/100%, respectively. The most area-efficient implementation, [DLK+25], uses about half our hardware resources but is more than 3× slower in signing and omits the key-generation functionality. Compared to [HLL+25], which exhibits the best area–time performance, our design uses 17.77% fewer LUTs and 3.06% fewer FFs. In BRAM usage, [HLL+25] employs a Group Subtree strategy for Merkle-tree computation that avoids storing an entire layer of a full tree, reducing BRAM to 48.48% of ours. However, our design requires an entire layer of tree nodes to be stored to be compatible with LMS/XMSS fast signatures. This is a necessary overhead due to architectural uniformity. In terms of Sign speed, the 8-unroll, 4-stage pipelined hash core and 180 MHz frequency in [HLL+25] make it $2 \sim 3\times$ faster than our 2-parallel, 2-unroll architecture. While there is a gap for standalone SPHINCS+, our core advantage is lower LUT/FF usage while flexibly supporting all security parameters of the three HBS

Table 7: Comparison of resource usage and performance across schemes.

Scheme	Param[b]	LUT/FF/BRAM/DSP	Freq (MHz)	Sign (ms)	Keygen (ms)	ATP[c]	
						sign	keygen
SPHINCS+ [ALCZ20]	$n = 128$	47,991/72,505/11.5/1	250	1.01	1.01	2.41	2.41
	$n = 192$	48,398/73,476/17.0/1		1.17	1.17	2.96	2.96
	$n = 256$	51,009/74,539/22.5/1		2.52	2.52	6.91	6.91
SPHINCS+ [HLL+25]	$n = 128$	29,410/14,090/4/0	179.8	1.96	0.082	1.97	0.08
	$n = 192$			3.19	0.12	3.21	0.12
	$n = 256$			6.61	0.316	6.64	0.32
SPHINCS+ [DLK+25]	$n = 128$	10,197/7,357/8/0	150	19.3	–	10.28	–
	$n = 192$	10,333/7,590/8/0		31.11	–	16.77	–
	$n = 256$	10,416/7,072/8/0		61.77	–	33.03	–
SPHINCS+ **[This work]**	$n = 128$	**24,184/13,659/16.5/0**	**108**	**4.95**	**0.22**	**5.73**	**0.25**
	$n = 192$			**8.52**	**0.32**	**9.88**	**0.37**
	$n = 256$			**17.75**	**0.84**	**20.57**	**0.98**
LMS [SHTW23]	$h, w = 5, 1$	139,886/103,914/58/3	267	0.0022	0.21	0.01	1.17
	$w = 1$	–		0.0022	1.81	–	–
	$w = 2$	–		0.0021	0.92	–	–
	$w = 4$	–		0.0037	0.89	–	–
	$w = 8$	–		0.0264	7.09	–	–
LMS [THG24]	$w = 4$ Min	15,860/13,914/15/0	100	1.18	785	0.97	647.55
	$w = 4$ Med	26,828/19,515/15/0		0.34	223	0.37	245.57
	$w = 4$ Lar	36,534/24,197/15/0		0.22	140	0.24	154.17
LMS **[This work]**	$w = 2$	**24,184/13,659/16.5/0**	**108**	**0.03**	**35.04**	**0.04**	**40.60**
	$w = 4$			**0.05**	**53.76**	**0.06**	**62.30**
XMSS [THG24]	$w = 4$ Min	15,860/13,914/15/0	100	6.9	4,680	5.88	3,985.55
	$w = 4$ Med	26,828/19,515/15/0		1.6	1,100	1.84	1,267.27
	$w = 4$ Lar	36,534/24,197/15/0		1.2	783	1.69	1,104.98
XMSS [CWW+22]	$w = 2$	22,300/31,215/0/0	110	5.54	1,799.82	5.25	1,706.16
	$w = 4$			11.09	2,881.53	10.51	2,731.58
XMSS **[This work]**	$w = 2$	**24,184/13,659/16.5/0**	**108**	**0.07**	**72.03**	**0.08**	**83.47**
	$w = 4$			**0.13**	**144.28**	**0.16**	**167.19**

[a] Dashes (-) indicate completely unavailable data.

[b] Unless otherwise specified, the data for LMS and XMSS are selected with $h = 10$ for fair comparison. Remaining parameters for each configuration (parameter n for SPHINCS+ and h, w for LMS/XMSS) can be found in Table 1 and Table 2, respectively.

[c] ATP is calculated as equivalent Slices $\times$ time. Equivalent Slice is calculated as $LUT/4 + FF/8 + BRAM \times 232.4 + DSP \times 102.4$ for 7 series and $LUT/4 + FF/8 + BRAM \times 116.2 + DSP \times 51.2$ for Ultrascale+.

algorithms, yielding greater flexibility and scalability.

For LMS, [SHTW23] proposes an ultra-high-throughput implementation scheme with 34 parallel hash modules for OTS chain computation on Virtex UltraScale+ FPGA. To allow many hash modules to flexibly process diverse inputs at different stages, the design requires complex control logic and large on-chip storage. This results in very high resource usage of approximately 140k LUTs, 104k FFs, and 58 BRAMs. This remains the case for the minimum parameter setting of $\{h, \omega\} = 5, 1$, and the required resources increase as the parameter increases. In comparison, our design reduces LUT/FF/DSP usage by 82.71%/86.86%/100%, respectively. For XMSS, the hardware implementation presented in [CWW+22] is partitioned into separate modules for key generation, signature, and verification (the hardware resource consumption of that work in the table corresponds to the signature module alone, with key generation requiring 8464 LUTs and 14464 FFs), whereas our design integrates the first two operations and delivers $82.11\times$ /$24.99\times$ and $82.85\times$ /$19.97\times$ speedups for Sign/Keygen when $h, \omega = 10, 2$ and $10, 4$, respectively. In terms of ATP, the design in [CWW+22] incurs $67.15\times$ /$20.44\times$ and $67.77\times$ /$16.34\times$ higher ATP than this work for Sign/Keygen under $h, \omega = 10, 2$ and $10, 4$, respectively.

5 Conclusion

To the best of our knowledge, this work introduces the first efficient and configurable accelerator that supports both stateful (LMS/XMSS) and stateless (SPHINCS+) hash-based signature schemes within a unified architecture. High throughput is achieved through a dual-parallel unroll-2 engine and a customized high-speed input path for the high-frequency OTS task. Meanwhile, area efficiency is maximized by reusing control logic via task abstraction and a hierarchical counter system, and is further optimized by a flexible padding shifter that accommodates diverse task inputs, thereby eliminating the need for large-scale multiplexers. Furthermore, memory efficiency is optimized by overlapping BRAM access with hash execution to conceal latency, and by employing segmented signature streaming to minimize peak memory footprint through real-time data output. Overall, the proposed design achieves a favorable performance-area trade-off and delivers significant acceleration for LMS/XMSS.

In future work, we will conduct an in-depth side-channel analysis of the current design. This investigation will extend beyond theoretical modeling to include practical attack simulations, allowing us to identify and characterize potential vulnerabilities related to power consumption and electromagnetic emissions. Based on these findings, we plan to develop and integrate a multi-layered suite of countermeasures, from hardware-level masking to algorithmic randomization. The objective is to effectively mitigate information leakage, thereby enhancing the system's overall end-to-end security. What's more, the scalable task abstraction architecture allows our design to extend support to other HBS algorithms with minimal overhead, providing a single, efficient accelerator for all NIST-standardized hash-based signatures, which in turn simplifies deployment.

References

[ALCZ20] Dorian Amiet, Lukas Leuenberger, Andreas Curiger, and Paul Zbinden. FPGA-based SPHINCS$^+$ Implementations: Mind the Glitch. In *23rd Euromicro Conference on Digital System Design, DSD 2020, Kranj, Slovenia, August 26-28, 2020*, pages 229–237. IEEE, 2020.

[BDH11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. XMSS-a practical forward secure signature scheme based on minimal security assumptions.

In *International Workshop on Post-Quantum Cryptography*, pages 117–129. Springer, 2011.

[BHK+19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.

[BHRvV21] Joppe W Bos, Andreas Hülsing, Joost Renes, and Christine van Vredendaal. Rapidly verifiable XMSS signatures. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 137–168, 2021.

[CAD+20] David A. Cooper, Daniel C. Apon, Quynh H. Dang, et al. Recommendation for Stateful Hash-Based Signature Schemes. NIST Special Publication 208, National Institute of Standards and Technology, 2020.

[CCJ+16] Lily Chen, Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray A Perlner, and Daniel Smith-Tone. *Report on post-quantum cryptography*, volume 12. US Department of Commerce, National Institute of Standards and Technology . . . , 2016.

[CKRS20] Fabio Campos, Tim Kohlstadt, Steffen Reith, and Marc Stöttinger. Lms vs xmss: comparison of stateful hash-based signature schemes on arm cortex-m4. In *International Conference on Cryptology in Africa*, pages 258–277. Springer, 2020.

[CWW+22] Yuan Cao, Yanze Wu, Wen Wang, Xu Lu, Shuai Chen, Jing Ye, and Chip-Hong Chang. An Efficient Full Hardware Implementation of Extended Merkle Signature Scheme. *IEEE Trans. Circuits Syst. I Regul. Pap.*, 69(2):682–693, 2022.

[DF17] Luca De Feo. Mathematics of Isogeny-Based Cryptography. *arXiv preprint arXiv:1711.04062*, 2017.

[DLK+25] Sanjay Deshpande, Yongseok Lee, Cansu Karakuzu, Jakub Szefer, and Yunheung Paek. SPHINCSLET: An Area-Efficient Accelerator for the Full SPHINCS+ Digital Signature Algorithm. *ACM Trans. Embed. Comput. Syst.*, 24(5):69:1–69:19, 2025.

[DS05] Jintai Ding and Dieter Schmidt. Rainbow, a New Multivariable Polynomial Signature Scheme. In *International Conference on Applied Cryptography and Network Security*, pages 164–175, Berlin, Heidelberg, 2005. Springer.

[HBG+18] Andreas Hülsing, Denis Butin, Stefan Gazdag, et al. RFC 8391: XMSS: eXtended Merkle Signature Scheme. RFC Editor, 2018.

[HLL+25] Tianze Huang, Jiahao Lu, Dongsheng Liu, Zhixiang Luo, Chi Cheng, Aobo Li, Lei Chen, Shuo Yang, Jiaming Zhang, and Xiang Li. An Efficient and Reconfigurable Post-Qantum Crypto-Processor for SPHINCS+. *IEEE Trans. Circuits Syst. I Regul. Pap.*, 72(5):2252–2262, 2025.

[KGC+20] Vinay B. Y. Kumar, Naina Gupta, Anupam Chattopadhyay, Michael Kasper, Christoph Krauß, and Ruben Niederhagen. Post-Quantum Secure Boot. In *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*, pages 1582–1585, 2020.

[MAB+18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (HQC). *NIST PQC Round*, 2(4):13, 2018.

[MCF19] David McGrew, Michael Curcio, and Scott Fluhrer. RFC 8554: Leighton-Micali Hash-based Signatures. RFC Editor, 2019.

[NIS22] Stateless Hash-Based Digital Signature Standard. FIPS 205, National Institute of Standards and Technology, 2022. Initial Public Draft.

[NIS23a] Module-Lattice-Based Digital Signature Standard. FIPS 204, National Institute of Standards and Technology, 2023.

[NIS23b] Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, National Institute of Standards and Technology, 2023.

[Por13] Thomas Pornin. Deterministic usage of the digital signature algorithm (DSA) and elliptic curve digital signature algorithm (ECDSA). Technical report, 2013.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. volume 21, pages 120–126. ACM, 1978.

[Saa24] Markku-Juhani O. Saarinen. Accelerating SLH-DSA by Two Orders of Magnitude with a Single Hash Unit. Cryptology ePrint Archive, Paper 2024/367, 2024.

[SHTW23] Yifeng Song, Xiao Hu, Jing Tian, and Zhongfeng Wang. A High-Speed FPGA-Based Hardware Implementation for Leighton-Micali Signature. *IEEE Trans. Circuits Syst. I Regul. Pap.*, 70(1):241–252, 2023.

[THG24] Jan Philipp Thoma, Darius Hartlief, and Tim Güneysu. Agile Acceleration of Stateful Hash-based Signatures in Hardware. *ACM Trans. Embed. Comput. Syst.*, 23(2):29:1–29:29, 2024.

[WJW+18] Wen Wang, Bernhard Jungk, Julian Wälde, Shuwen Deng, Naina Gupta, Jakub Szefer, and Ruben Niederhagen. XMSS and Embedded Systems - XMSS Hardware Accelerators for RISC-V. Cryptology ePrint Archive, Paper 2018/1225, 2018.

[WOS24] A. Wagner, F. Oberhansl, and M. Schink. Extended version—to be, or not to be stateful: post-quantum secure boot using hash-based signatures. *Journal of Cryptographic Engineering*, 14:631–648, 2024.